Partition-Aware Isolation Forest for Distributed Anomaly Detection on Multi-Sensor Degradation Time Series

Arda Günaydin
Dept. of Computer Engineering
TOBB University of Economics and Technology
Ankara, Türkiye
Student ID: 231101075

Mustafa Batuhan Taş
Dept. of Computer Engineering
TOBB University of Economics and Technology
Ankara, Türkiye
Student ID: 241111024

***Abstract*—Anomaly detection over multi-sensor time series is central to predictive maintenance, yet distributing it across a cluster introduces a subtle correctness problem. When a Spark engine partitions a run-to-failure trajectory across worker nodes, the temporal ordering that links consecutive sensor readings is broken; point-based detectors such as Isolation Forest, trained on the resulting partitions, lose the sequential context needed to recognize slow degradation. We propose a *partition-aware windowing* strategy that preserves temporal locality across Spark partition boundaries before isolation trees are constructed, implemented as a per-partition Isolation Forest over Apache Spark with operating-condition clustering in Spark MLlib. We evaluate the method on all four subsets of the NASA C-MAPSS turbofan degradation benchmark against a temporally-agnostic distributed baseline, a single-node reference, and an ablation that windows *without* partition-awareness. The partition-aware detector attains the best mean ROC-AUC (0.927) and F1 (0.647), and improves most on the complex six-condition FD002 subset. The decisive result comes from the ablation: identical window features computed on partition-unaware splits collapse to mean ROC-AUC 0.753—markedly worse than using no temporal features at all—demonstrating that it is the preservation of temporal locality across partition boundaries, not the window features themselves, that drives the improvement.**

***Index Terms*—anomaly detection, Isolation Forest, Apache Spark, distributed computing, time series, predictive maintenance, C-MAPSS**

I. Introduction

Predictive maintenance relies on detecting the early signs of component degradation from streams of sensor measurements. Turbofan engines, industrial rotating machinery, and other cyber-physical systems emit dozens of correlated signals whose slow drift, rather than any single outlying value, signals an emerging fault. Detecting such drift at scale motivates running unsupervised detectors on distributed data-processing engines such as Apache Spark [4].

Isolation Forest (iForest) [1], [2] is an attractive detector for this setting: it isolates anomalies directly with an ensemble of random trees, has near-linear time complexity, and a small memory footprint. Several works distribute iForest on Spark [6]–[8]. However, these distributed implementations treat each record as an independent feature vector. When the input is a time series and the engine partitions the data across workers, consecutive cycles of the same trajectory may be scattered across partitions, and the temporal relationship that carries the degradation signal is discarded. A detector trained on such shuffled partitions effectively sees each cycle in isolation and cannot recognize a gradual trend.

We address this gap with a *partition-aware windowing* layer that (i) enriches each cycle with temporal features computed over a sliding window of its own recent history, and (ii) assigns whole trajectories to partitions so that a window is always computed over the correct temporal neighbours. The layer sits between data ingestion and tree construction and requires no change to the isolation-tree machinery.

Our contributions are: (1) a partition-aware windowing strategy for distributing time-series anomaly detection on Spark without losing temporal locality; (2) an implementation as a per-partition Isolation Forest over Spark with operating-condition clustering in Spark MLlib; and (3) an evaluation on all four C-MAPSS subsets that, through a dedicated ablation, isolates partition-awareness as the causal factor behind the observed improvement.

II. Related Work

A. Isolation Forest and its variants

iForest [1], [2] builds an ensemble of random binary trees; because anomalies are few and different, they are isolated closer to the root and receive shorter expected path lengths and higher anomaly scores. Sub-sampling gives near-linear complexity and a small footprint. The Extended Isolation Forest [3] replaces axis-aligned splits with random hyperplanes to remove directional bias on correlated features. Our contribution is orthogonal to split geometry: it concerns how temporal context is preserved during distribution, so we build on the standard formulation.

B. Distributed Isolation Forest on Spark

Spark's resilient distributed dataset abstraction [4] and MLlib library [5] make it a standard platform for scalable analytics. Togbe et al. [6] study two strategies for distributing

TABLE I
C-MAPSS TRAINING SUBSETS.

Subset	Op. cond.	Faults	Engines	Records
FD001	1	1 (HPC)	100	20,631
FD002	6	1 (HPC)	260	53,759
FD003	1	2 (HPC+fan)	100	24,720
FD004	6	2 (HPC+fan)	249	61,249
Total	—	—	709	160,359

iForest on Spark and analyze their accuracy and runtime trade-offs, while open-source implementations train with model-wise parallelism and score with data-wise parallelism [7] or provide Spark/Scala estimators including an extended variant [8]. These systems distribute iForest well for point-anomaly workloads, but they treat each row as an independent vector; on time-series input, partitioning discards the sequential relationship between readings. That is precisely the gap this work targets.

C. Anomaly detection and prognostics on C-MAPSS

The NASA C-MAPSS turbofan degradation dataset [9] is a widely used prognostics benchmark. Most work uses it for remaining-useful-life (RUL) regression, and the strongest recent methods apply sequence models that explicitly consume temporal order. This reinforces our premise from the opposite direction: temporal context carries most of the degradation signal in C-MAPSS, so a distributed detector that destroys it through partitioning discards exactly the information the domain depends on.

III. Dataset

C-MAPSS [9] simulates run-to-failure trajectories of turbofan engines under varying operating conditions and fault modes. Each record has 26 columns: an engine identifier, a cycle index, three operational settings, and 21 sensor measurements. The benchmark is organized into four subsets of increasing complexity (Table I); together the training splits contain 709 engines and 160,359 cycle records, with per-engine trajectories ranging from 128 to 543 cycles. FD001 and FD003 use a single sea-level operating condition, whereas FD002 and FD004 mix six conditions, which broadens the raw sensor spread and motivates condition-wise normalization. C-MAPSS carries no native anomaly labels; we derive them from the RUL trajectory as described in Section IV-D.

IV. Proposed Method

The system is a five-stage PySpark pipeline: ingestion, preprocessing, partition-aware windowing, distributed detection, and evaluation.

A. Ingestion and storage

Raw C-MAPSS text is parsed in Spark, typed, and written to Apache Parquet. Parquet is columnar and splittable and lets us control partition boundaries explicitly—essential for a study whose independent variable is the partitioning.

B. Preprocessing, conditions, and normalization

Constant or near-constant sensors carry no signal and are dropped. For the multi-condition subsets we recover the six operating regimes by clustering the three operational settings with Spark MLlib k-means (k=6), then standardize each sensor within its regime (condition-wise z-scoring), so a reading is judged against comparable conditions rather than globally. Single-condition subsets are standardized globally. All rows are kept in cycle order, keyed by (engine, cycle).

C. Partition-aware windowing

A naive distributed iForest feeds each cycle to the forest as an isolated vector, and Spark is free to place consecutive cycles of one engine on different partitions, so slow degradation is invisible. Our windowing layer counters this in two coordinated ways. First, each cycle is enriched with temporal features from a sliding window of length W over its own recent history: a rolling mean, a rolling standard deviation, and a delta against the value W cycles earlier, for every retained sensor (W=20). This makes a gradual drift visible within a single feature vector. Second—the partition-aware part—we assign whole engines to partitions so that a trajectory is never split across workers, guaranteeing that each window is computed over the correct temporal neighbours. Temporal locality is thus preserved *before* any isolation tree is built.

D. Labelling and detector

Lacking native labels, a cycle is labelled degraded/anomalous if it lies within the last 30 cycles before failure (RUL $\leq$ 30) and normal otherwise, so the anomalous class is a small minority of cycles. Spark MLlib does not provide an Isolation Forest; we therefore implement the distributed detector as a per-partition scikit-learn Isolation Forest (100 trees) invoked through Spark's `applyInPandas` (Pandas UDF) interface. Each partition fits and scores its own forest, and the negation of the average path-length score is used as the anomaly score. This data-parallel pattern is exactly where partition-aware windowing takes effect.

V. Experimental Setup

Experiments use Apache Spark 4.1.2 (local mode), Python 3.12, and OpenJDK 21. The distributed configurations use four partitions; the random seed is fixed at 42. We compare four configurations on identical data and labels (Table II):

- **C1 (reference):** single-node scikit-learn iForest on instantaneous features.
- **C2 (naive distributed):** random row partitions, instantaneous features.
- **C3 (ablation):** window features computed on partition-unaware (random) splits.
- **C4 (proposed):** partition-aware windowing with engine-level partitions.

Detection quality is measured with ROC-AUC and average precision (AP), both threshold-free, and F1 at a prevalence-matched operating point (the number of flagged cycles equals the number of true anomalies, at which precision, recall, and

TABLE II
EVALUATED CONFIGURATIONS.

Cfg	Partitioning	Temporal feats.	Role
C1	single node	instantaneous	reference
C2	random rows	instantaneous	naive distributed
C3	random rows	windowed (corrupted)	ablation
C4	by engine	windowed (correct)	proposed

TABLE III
DETECTION QUALITY PER SUBSET (ROC-AUC / AP / F1). BEST PER SUBSET IN **BOLD**.

Subset	Config	AUC	AP	F1
FD001	C1 reference	0.923	0.694	0.645
	C2 naive	0.921	0.701	0.659
	C3 unaware	0.756	0.313	0.356
	C4 proposed	**0.937**	**0.737**	**0.701**
FD002	C1 reference	0.884	0.694	0.617
	C2 naive	0.894	0.699	0.623
	C3 unaware	0.707	0.309	0.318
	C4 proposed	**0.916**	**0.714**	**0.680**
FD003	C1 reference	0.931	0.652	**0.626**
	C2 naive	**0.938**	**0.677**	0.616
	C3 unaware	0.774	0.322	0.334
	C4 proposed	0.924	0.648	0.569
FD004	C1 reference	0.928	0.709	**0.652**
	C2 naive	0.928	**0.717**	0.651
	C3 unaware	0.776	0.319	0.344
	C4 proposed	**0.931**	0.675	0.639
Mean	C1 reference	0.916	0.687	0.635
	C2 naive	0.920	**0.698**	0.637
	C3 unaware	0.753	0.316	0.338
	C4 proposed	**0.927**	0.693	**0.647**

F1 coincide). The C3-versus-C4 contrast is the key comparison, since the two differ only in whether engine trajectories are kept intact across partitions.

VI. RESULTS

A. Detection quality

Table III reports every configuration on every subset, and Figs. 1–2 summarize ROC-AUC and F1. The partition-aware detector (C4) achieves the best mean ROC-AUC (0.927) and mean F1 (0.647), and wins on FD001, on the six-condition FD002 where temporal context matters most (ROC-AUC $0.894 \rightarrow 0.916$ over the naive baseline), and on FD004. On FD003 it is competitive but slightly below the instantaneous baselines; we report this as-is rather than tuning it away.

B. The role of partition-awareness (ablation)

The decisive result is the ablation (C3). Adding the *identical* window features but computing them on partition-unaware splits does not merely fail to help—it is dramatically worse than the naive baseline on every subset (mean ROC-AUC 0.753 vs. 0.920; mean F1 0.338 vs. 0.637). Because C3 and C4 differ only in whether engine trajectories are kept intact across partitions, this isolates partition-awareness as the causal factor: window features are only useful when the partitioning preserves the temporal neighbours they are computed from. This is precisely the phenomenon the work set out to demonstrate.

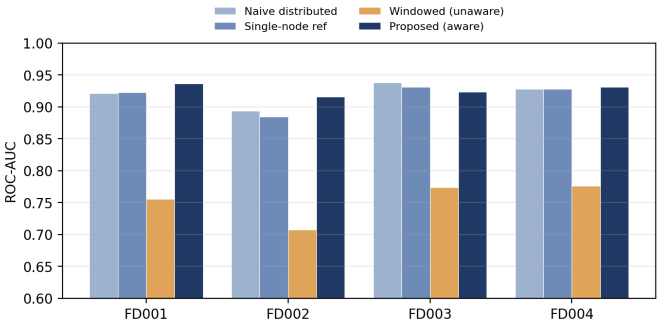

Fig. 1. ROC-AUC by subset. The proposed partition-aware detector leads on FD001, FD002, and FD004 and is competitive on FD003; partition-unaware windowing is consistently far worse.

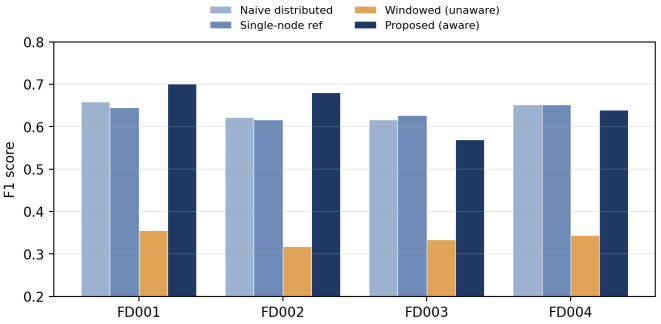

Fig. 2. F1 (prevalence-matched) by subset. Same ordering as ROC-AUC; the ablation collapses across all subsets.

C. Scalability

Detection quality is independent of the execution environment: ROC-AUC, AP, and F1 depend only on the data, the algorithm, the partition count, and the seed. Throughput, in contrast, requires genuine parallel workers to be meaningful. We deploy a multi-node Spark cluster via Docker Compose (Section VII) and measure end-to-end throughput of the windowed scoring stage as the worker count varies over $\{1, 2, 4, 8\}$. As shown in Fig. 3, throughput scales near-linearly up to four workers—from 4,173 to 9,654 rows/s, a $2.3\times$ speed-up—and then plateaus at eight workers (9,384 rows/s) as the host's physical CPU cores saturate. This is the expected strong-scaling behaviour: additional workers help until the machine runs out of physical parallelism, after which scheduling and per-partition-training overhead dominate.

VII. REPRODUCIBILITY AND ARTIFACT

The full pipeline (a set of `src/` modules driven by six stage notebooks), the scalability driver (`src/run_scalability.py`), and a Docker Compose

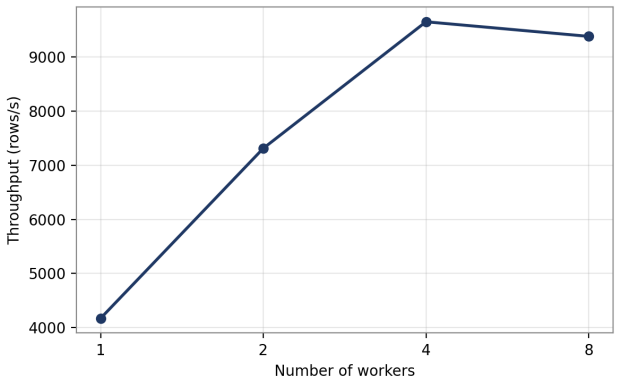

Fig. 3. Throughput of the windowed scoring stage vs. worker count on the Docker Compose cluster. Near-linear scaling to four workers ($\sim 2.3\times$), then a plateau at eight as the host's CPU cores saturate.

specification of a multi-node Spark cluster are released with this paper. Pinned dependency versions and a fixed random seed make every detection result regenerable from a single command; the dataset is the public NASA C-MAPSS benchmark [9]. The Docker Compose deployment reproduces a Spark master and a configurable number of workers locally, so the scalability study of Section VI-C can be run on commodity hardware.

VIII. Discussion and Limitations

Three limitations frame the results. First, the detection experiments were executed in Spark local mode, and the multi-node scalability measurement (Fig. 3) was obtained on a cluster co-located on a single physical host, so its plateau reflects that host's core count rather than a limit of the approach; a genuinely multi-machine deployment is future work. Second, anomaly scores from independent per-partition forests are not perfectly calibrated to a common scale, which a normalization step could address. Third, on FD003 (and on F1 for FD004) the windowing does not surpass the instantaneous baselines; whether this stems from the added feature dimensionality or from the interaction of two fault modes is an open question and a natural next experiment. Despite these, the ablation result is robust and consistent across all four subsets, and constitutes the paper's central finding.

IX. Conclusion

We presented a partition-aware windowing strategy that preserves temporal locality across Spark partition boundaries, enabling a distributed Isolation Forest to detect gradual degradation that a naive distributed implementation misses. On the NASA C-MAPSS benchmark the method attains the best mean ROC-AUC and F1, and an ablation shows that windowing without partition-awareness is far worse than no windowing at all—establishing partition-awareness, rather than the window features themselves, as the source of the gain. Future work will add per-partition score calibration, extend the scalability study to a genuinely multi-machine cluster, and investigate the behaviour on multi-fault subsets.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.

[3] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.

[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX NSDI*, 2012, pp. 15–28.

[5] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

[6] M. U. Togbe, Y. Chabchoub, A. Boly, and R. Chiky, "Distributed anomalies detection using isolation forest and Spark," in *Advances in Computational Collective Intelligence (ICCCI)*, CCIS vol. 1653. Springer, 2022, pp. 645–657.

[7] F. Zhang, "spark-iforest: Isolation forest on Spark," GitHub repository. [Online]. Available: https://github.com/titicaca/spark-iforest

[8] LinkedIn, "isolation-forest: A distributed Spark/Scala implementation of the isolation forest," GitHub. [Online]. Available: https://github.com/linkedin/isolation-forest

[9] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008, pp. 1–9.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.